Technical Design Document

Medical Device Rental and Inventory Management System

Prepared for: Mahesh Foundation

Date: 06/04/2026

This document presents architectural flow & two tech stack approaches for the proposed system, with detailed resource breakdown, pros & cons, and annual cost estimates on Azure to support an informed decision.

1. Executive Summary

This document outlines two architectural options for building the Medical Device Rental & Inventory Management System for Mahesh Foundation. Each approach has been evaluated against the requirements in the Business Requirements Document (BRD), with considerations for scalability, cost, ease of build, and long-term maintainability. Both options are assessed with Azure as the hosting platform.

.

Option	Description	Best Suited For
Option A	WordPress + Plugin Ecosystem + Integrations	Teams preferring a familiar CMS platform
Option B	**Lovable (Next.js) + Supabase — Modern Custom Build**	**Scalable, mobile-ready— Recommended**

2. System Overview

The Medical Device Rental & Inventory Management System will serve the following primary functions:

- Centralized asset and inventory management across multiple centers
- End-to-end lease lifecycle management — request, approval, issuance, return
- Repair and warranty tracking
- Role-based access for Admin, Center Admin, and Approver roles
- Automated SMS notifications via MSG91 for all workflow triggers
- QR / barcode-based device identification and scanning
- Dashboards, reports, and utilization analytics
- Public-facing lease request portal for requestors — no login required

User Roles

Role	Access Level	Key Functions
Admin	Full system access	All functions including user management and reports
Center Admin	Center-level access	Inventory, issuance, returns, asset transfers
Approver	Approval workflows only	Review and approve or reject lease requests
Public	No login required	Submit lease requests and track status via token

SYSTEM WORKFLOW DIAGRAM

1 LAYER 1 USERS

People who interact with the system

PUBLIC USER

- No login
- Submits request
- Receives token
- Tracks status

INTERNAL USERS

- Login via OTP
- Role determines access:
  - Center Role → Operational actions
  - Approver → Decision-making
  - Admin → Full control

2 LAYER 2 INTERFACE LAYER

How users interact with the system

PUBLIC INTERFACE

- Request submission
- Token tracking

LOGIN INTERFACE

OTP

- OTP verification
- Secure login

INTERNAL SYSTEM

- Single system
- Views change dynamically based on role

3 LAYER 3 PROCESSING LAYER

Business logic and workflows

HANDLES CORE LOGIC

- Token Generation for Requests
- OTP Handling for Login
- Workflow Transitions (Approve/Reject/ Issue/Return)
- Notification Triggers
- Scheduled Reminders

4 LAYER 4 DATA LAYER

Stores and manages system data

STORES

- Users
- Requests
- Assets
- Transactions

ENFORCES

- Role-based Access
- Data Visibility Rules

MAINTAINS

- Real-time Updates
- Audit Logs

5 LAYER 5 EXTERNAL INTERACTION LAYER

Connects with external services and processes

HANDLES EXTERNAL INTERACTIONS

MESSAGE DELIVERY
(OTP, Alerts, Reminders)

COMPLIANCE REQUIREMENTS
(Policies, Regulations, Data Protection)

DEPLOYMENT & UPDATES
(System Releases, Patches, Scaling)

Layer-by-Layer Workflow

Layer 1 — Users

- **Public User**
    - No login
    - Submits request
    - Receives token
    - Tracks status
- **Internal Users**
    - Login via OTP
    - Role determines access:
        - Center role → operational actions (issuance, returns, asset transfer)
        - Approver → decision-making (review- approve/reject – add comments)
        - Admin → full control

Layer 2 — Interface Layer

- Public interface:
    - SKU Catalog
    - Request submission
    - Token tracking
- Login interface:
    - OTP verification
- Internal system:
    - Single system
    - Views change dynamically based on role

Layer 3 — Processing Layer

Handles core logic:

- Token generation for requests
- OTP handling for login
- Workflow transitions (approve/reject/issue/return)
- Notification triggers

- Scheduled reminders

Layer 4 — Data Layer

- Stores:
    - Users
    - Requests
    - Assets
    - Transactions
- Enforces:
    - Role-based access
    - Data visibility rules
- Maintains:
    - Real-time updates
    - Audit logs

Layer 5 — External Interaction Layer

- Handles:
    - Message delivery (OTP, alerts, reminders)
    - Compliance requirements
    - Deployment & updates

3. Suggested Tech Stacks:

(3.a) Option A — WordPress + Integrations

WordPress is used as the core platform, with custom plugins and third-party integrations handling workflow, SMS, and QR functionality. This is a familiar platform but carries significant overhead for a workflow-driven application of this nature.

Technology Stack

Layer	Technology	Purpose
Frontend & Backend	WordPress (PHP)	Core platform, UI, admin panel
Database	Azure MySQL Flexible Server	All data storage
Forms & Workflow	Gravity Forms + custom PHP	Lease requests, approval flows
Custom Data	Advanced Custom Fields (ACF Pro)	Asset fields, serial numbers, warranty details
User Roles	User Role Editor Plugin	Role-based access control
SMS	MSG91 via REST API	All notification and reminder triggers
QR Scanning	html5-qrcode JS library	Browser-based barcode scanning
Scheduled Jobs	Server-side cron (not WP-Cron)	Return reminders and overdue alerts
Hosting	Azure App Service + Azure MySQL	Production environment on Azure

What WordPress Handles Well vs. What It Struggles With

Handles Well (60–70%)	Requires Heavy Custom Work (30–40%)
✓ Asset and inventory management via custom post types	✗ Complex multi-stage workflows require custom state management and are not natively supported
✓ Basic user roles and access control	✗ Time-critical scheduling requires external cron; built-in scheduling is traffic-dependent and unreliable
✓ Form-based lease request submission	✗ Real-time data synchronization is not natively supported and requires external mechanisms
✓ Basic dashboards and tabular reports	✗ Advanced device integrations (camera scanning, hardware access) require custom frontend handling
✓ Return logging and status update forms	✗ Secure identity verification flows (e.g., Aadhaar) require external integrations and compliance handling Token generation is simple, but enforcing uniqueness and traceability at scale requires structured logic

Pros & Cons

Advantages	Disadvantages
✓ Widely known platform — easy to find developers	✗ Heavier architecture compared to custom-built systems — performance overhead at scale
✓ Large plugin ecosystem reduces initial build time	✗ Dependency on plugins — updates can introduce instability or conflicts
✓ Familiar admin UI for non-technical users	✗ Not inherently designed for workflow-driven or stateful applications

✓ Many hosting providers support it natively on Azure	✗ Mobile app expansion requires rebuilding core logic in a different stack
✓ Fast time-to-market for MVP and internal tools	✗ Increasing costs over time due to plugin licenses and scaling infrastructure
✓ Strong community support and documentation	✗ Higher long-term maintenance effort as dependencies and customizations grow

Annual Cost Estimate — Option A on Azure

Service	Specification	Monthly (INR)	Annual (INR)
Azure App Service (South India)	B2 tier — 2 cores, 3.5GB RAM	₹10,380.737	₹1,24,569(approx)
Azure MySQL Flexible Server	Burstable B1MS	₹ **1,695.521**	₹20,347 (approx)
Azure Blob Storage	100GB for media and assets	₹227	₹2730
Azure Front Door Standard (CDN) (Required only if the public usage scale is large, can be integrated in phase two)	Standard tier (CDN Classic retired Sep 2027)	₹3318.044(base) ₹1033(100gb data usage)	₹52,300(approx.) (range may vary based on data usage)
Plugin Licences	Gravity Forms(pro) + ACF Pro + WP Rocket	13000/year +4000/year+ 5000/year	₹22000
MSG91 SMS	~750 SMS/month @ ₹0.25/SMS	₹188-200	₹2,250
Domain + DLT Registration	Annual + one-time	—	₹2,500
TOTAL YEAR 1			**₹2,26,696**

(B2 gives us better control over data and workflows by reducing plugin dependency and separating concerns, which makes the system more maintainable and easier to scale compared to a tightly coupled B1 setup)

(These plugins i.e. Gravity forms, ACF pro, WP rocket are required to turn WordPress from a basic CMS into a structured, workflow-driven application with acceptable performance.)

(3.b) Option B — Lovable (Next.js) + Supabase ★ Recommended

A fully custom modern web application built using AI-assisted development (vibe coding) with Lovable as the primary build tool and Supabase as the backend-as-a-service platform. The frontend is hosted on Azure Static Web Apps while Supabase provides a managed, enterprise-grade PostgreSQL backend.

Technology Stack

Layer	Technology	Purpose
Frontend	Next.js + Tailwind CSS + shadcn/ui	Web application UI — all screens
Build Tool	Lovable.dev + Cursor AI	Assisted development tools used to accelerate UI and logic generation
Database	Supabase — PostgreSQL	All structured data — assets, leases, users, returns
Authentication	Supabase Auth — OTP/mobile	OTP login for all internal roles — no passwords
File Storage	Supabase Storage	Asset images, invoice documents, QR codes
Business Logic	Supabase Edge Functions	SMS triggers, token generation, cron jobs
Real-time Sync	Supabase Realtime	Near real-time updates for inventory changes across users
SMS	MSG91 REST API	All notification workflows — approvals, reminders, returns

QR Scanning	ZXing.js or Bluetooth scanner	Device identification on issuance and return
Frontend Hosting	Azure Static Web Apps	Production deployment on Azure
Future Mobile	React Native + Expo	Phase 2 mobile app — same codebase, no rebuild

Pros & Cons

Advantages	Disadvantages
✓ Lowest annual cost — less than half of Option A	✗ Hybrid cloud setup if other infrastructure is hosted on a different provider
✓ Modern, fast, scalable architecture	✗ Learning curve for the team (modern frameworks and backend services)
✓ Real-time inventory sync built into Supabase	✗ Requires engineering effort for custom workflows and integrations
✓ OTP authentication built in — no additional plugin	✗ Yearly Costs vary based on the usage
✓ Mobile app in Phase 2 extends naturally — no rebuild	✗ External messaging services (SMS) incur per-message charges that scale with usage
✓ Clean codebase — no plugin dependency risk	✗ Vendor dependency — pricing or policy changes can impact long-term costs

Annual Cost Estimate — Option B on Azure

Service	Specification	Monthly (INR)	Annual (INR)
Azure Static Web Apps	Standard tier — $9/month	₹853.212	₹10,236

Supabase Pro	PostgreSQL + Auth + Storage + Edge Functions — $25/month	₹2,318 (may vary based on usage)	₹27,816
MSG91 SMS	~750 SMS/month @ ₹0.25/SMS	₹188	₹2,250
Variable Costs(Supabase x Azure Hybrid)		₹500-₹2000	₹18000
Domain + DLT Registration	Annual + one-time	—	₹2,500
TOTAL YEAR 1			**₹60,802**

5. Option B — Side by Side Comparision

Factor	Option A — WordPress	Option B — Modern Stack
Annual Cost on Azure	₹2,26,696 (largely fixed)	₹60,802 baseline + usage-based scaling
Estimated Build Time	10–14 weeks	2-4 Weeks
Coding Required	Medium to High (custom PHP + plugins)	Low to Medium (assisted development + structured backend)
Performance	Moderate — plugin and theme overhead	High — lightweight, optimized architecture
Real-time Inventory Sync	Not native — requires workarounds	Built-in real-time capability
OTP Authentication	Plugin-dependent	Native support within auth system

Mobile App (Phase 2)	Requires separate backend or rebuild	Can reuse existing backend with minimal changes
Enterprise Readiness	Depends on plugin quality and setup	Backed by platform-level compliance standards
Long-term Maintenance	Plugin updates and compatibility risks	Lower dependency surface, but requires engineering ownership
Scalability	Suitable for moderate scale, may require re-architecture later	Designed to scale with increasing usage (cost scales accordingly)